

Challenge:	The Broken Howler
Category:	Steganography
Difficulty:	Easy
Tools Used:	HexEd.it, base64.sh, any browser (Chrome, Safari etc)

Summary

The challenge provides a corrupted file that cannot be opened or identified by standard tools, yet its structure suggests that the data inside is still intact. It hints that the issue is not with the content itself, but with how the file is being interpreted—specifically that its format signature has been altered. The goal is to recognize the correct file type, repair its identity, and then extract the hidden message.

In brief, the procedure involves inspecting the file at a low level using a hex viewer, identifying the incorrect magic bytes at the beginning, and replacing them with the correct signature to restore the original format. Once the file is repaired and opens successfully, the contents reveal an encoded message. This message is then decoded using Z85 decoding, ultimately recovering the original hidden flag.

Problem Statement

A Howler intercepted inside Chamber B-7 was never meant to reach its recipient. The Ministry flagged it as corrupted—unreadable, unclassifiable, and ultimately discarded. But the data inside was never destroyed.

Something in its outer structure has been tampered with—not erased, but altered just enough to prevent recognition. The message remains intact, sealed beneath a broken format.

If you can restore what was damaged, the Howler may still speak.

Steps to Solve

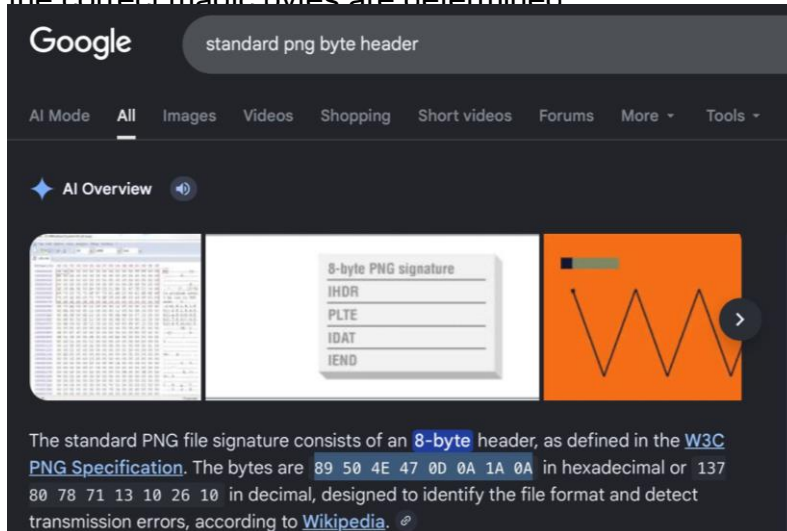
1. Upon opening the provided file (howler_fragment.owl), it fails to open or is misidentified, indicating possible corruption. Initial observation suggests the data is present, but the file is not being interpreted correctly (**Hint 1**).
2. Using a hex viewer, the beginning of the file is examined. The initial bytes (magic number) do not match any known file format, confirming that the file's signature has been altered (**Hint 2**).

```

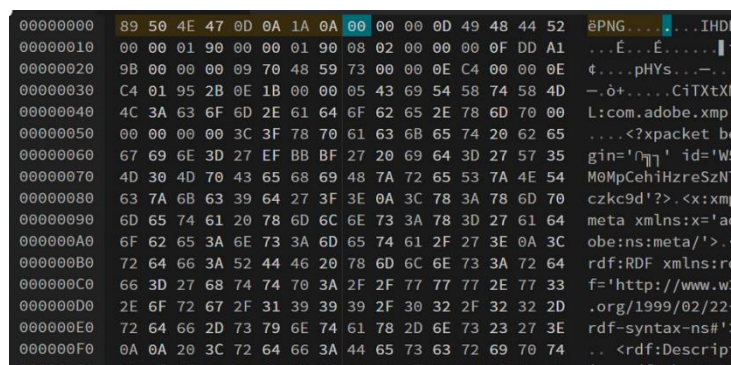
00000000 BA AD F0 0D DE AD BE EF 00 00 00 0D 49 48 44 52 i=...IHDR
00000010 00 00 01 90 00 00 01 90 08 02 00 00 00 0F DD A1 ...É...É...i
00000020 9B 00 00 00 09 70 48 59 73 00 00 0E C4 00 00 0E ...pHYs...-...
00000030 C4 01 95 2B 0E 1B 00 00 05 43 69 54 58 74 58 4D -.ò+....CiTXtXM
00000040 4C 3A 63 6F 6D 2E 61 64 6F 62 65 2E 78 6D 70 00 L:com.adobe.xmp.
00000050 00 00 00 00 3C 3F 78 70 61 63 6B 65 74 20 62 65 ...<?xpacket be
00000060 67 69 6E 3D 27 EF BB BF 27 20 69 64 3D 27 57 35 gin='&#x27; id='W5
00000070 4D 30 4D 70 43 65 68 69 48 7A 72 65 53 7A 4E 54 M0MpCehiHzreSzNT
00000080 63 7A 6B 63 39 64 27 3F 3E 0A 3C 78 3A 78 6D 70 czkc9d'?>.<x:xmp
00000090 6D 65 74 61 20 78 6D 6C 6E 73 3A 78 3D 27 61 64 meta xmlns:x='ad
000000A0 6F 62 65 3A 6E 73 3A 6D 65 74 61 2F 27 3E 0A 3C obe:ns:meta/'>.<
000000B0 72 64 66 3A 52 44 46 20 78 6D 6C 6E 73 3A 72 64 rdf:RDF xmlns:rd
000000C0 66 3D 27 68 74 74 70 3A 2F 2F 77 77 77 2E 77 33 f='http://www.w3
000000D0 2E 63 72 67 2F 31 39 39 39 2F 30 32 2F 32 32 2D .org/1999/02/22-
000000E0 72 64 66 2D 73 79 6E 74 61 78 2D 6E 73 23 27 3E rdf-syntax-ns#'>
000000F0 0A 0A 20 3C 72 64 66 3A 44 65 73 63 72 69 70 74 ..<rdf:Descript
00000100 69 6F 6E 20 72 64 66 3A 61 62 6F 75 74 3D 27 27 ion rdf:about=''
00000110 0A 20 20 78 6D 6C 6E 73 3A 41 74 74 72 69 62 3D . xmlns:Attrib=

```

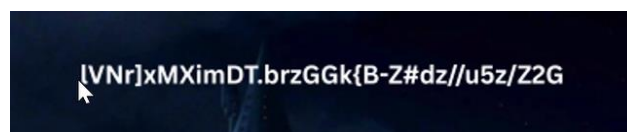
3. If we focus on the first few bytes, as guided by **Hint 3**, we identify that the header is incorrect. Search for the “standard png byte header” on any browser. By comparing with standard file signatures, the correct magic bytes are determined



4. The corrupted bytes are replaced with the correct file signature, restoring the file's original format and allowing it to open successfully.



5. Once opened, the file reveals an encoded string instead of readable content. This indicates that the data is still transformed and requires further processing (**Hint 4**).



6. The extracted string is identified as Z85 encoded data. Each group of characters represents encoded binary information.
7. The encoded string is then decoded using Z85 decoding, converting it back into its original byte form.



8. Repeating the decoding process for the entire string successfully reveals the hidden message and the final flag: **`Cruxhunt{burnt_but_not_lost}`**.